

PhysForge: Generating Physics-Grounded 3D Assets for Interactive Virtual World

Anonymous Authors¹



Figure 1. PhysForge, takes only a single input image to generate Physics-Grounded 3D Assets. The figure showcases our high-quality generated results, where: (a) parts are distinguished by different colors to show their geometry; (b) parts are annotated with its detailed physical properties (text labels); and (c) the kinematic parameters for movable parts (such as joint axes) are precisely indicated by arrows. Our assets are ready for building Interactive Virtual World.

Abstract

Synthesizing physics-grounded 3D assets is a critical bottleneck for interactive virtual worlds and embodied AI. Existing methods predominantly focus on static geometry, overlooking the functional properties essential for interaction. We propose that interactive asset generation must be rooted in functional logic and hierarchical physics. To bridge this gap, we introduce PhysForge, a

decoupled two-stage framework supported by PhysDB, a large-scale dataset of 150,000 assets with four-tier physical annotations. First, a VLM acts as a “physical architect” to plan a “*Hierarchical Physical Blueprint*” defining material, functional, and kinematic constraints. Second, a physics-grounded diffusion model realizes this blueprint by synthesizing high-fidelity geometry alongside precise kinematic parameters via a novel *KineVoxel Injection* (KVI) mechanism. Experiments demonstrate that PhysForge produces functionally plausible, simulation-ready assets, providing a robust data engine for interactive 3D content and embodied agents.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

1. Introduction

Recently, 3D generative models have achieved rapid progress, capable of synthesizing 3D assets with diverse appearances and high-fidelity geometric details (Zhang et al., 2023; Xiang et al., 2024). Concurrently, embodied AI and virtual game environments face a soaring demand for large-scale, high-quality 3D content. 3D generation technology holds the promise to serve as a *data engine* to this content bottleneck. However, a significant gap remains: the vast majority of existing 3D generation methods focus solely on generating *static* geometry and textures, overlooking the physics information that is crucial for interaction. These generated “hollow shell” assets cannot be grasped, pushed, or manipulated by agents, making them difficult to deploy directly in embodied AI simulators or game environments that require realistic physical interactions. To bridge this gap, we aim to propose a generation pipeline capable of producing *physics-grounded* 3D assets directly. Our core insight is that for an object to be physically interactive, its generation must be driven by its *functional logic* and *hierarchical physics*. For example, a button on a television is the basic unit of function and operation; a cabinet’s door and handle each carry distinct materials, functions, and kinematic definitions. Therefore, we shift the focus from traditional holistic shape generation to physics-centric synthesis, where the object’s structure is a manifestation of its intended physical functions.

To achieve this, we propose **PhysForge**, an innovative two-stage framework that decouples physical planning from physical realization. Inspired by the “planning-then-generation” paradigms successful in 2D multimodal research (Sun et al., 2024; Chen et al., 2025a), our design leverages the complementary strengths of specialized generative architectures: while VLMs possess the world knowledge necessary for complex physical planning, diffusion models excel at the precise synthesis of kinematic parameters, geometry, and textures. By decoupling these processes, PhysForge ensures that the generated assets are not only visually realistic but also physically consistent and simulation-ready.

The first stage is **VLM-based Planning**. Instead of starting from scratch, we finetune a powerful VLM, enabling it to acquire 3D spatial understanding and part-structure planning capabilities while retaining its inherent world knowledge. This VLM takes an image, an optional 2D mask, and generated 3D voxels (Xiang et al., 2024) as input, and is tasked with generating what we call “*Hierarchical Physical Blueprints*”. This blueprint includes the bounding box layout for all parts, as well as detailed physical properties for each part (including parent nodes, articulation types, etc.). We discover a critical synergistic effect: the introduction of physical properties, in turn, significantly aids the

model’s structural planning. By providing functional and physical constraints, it effectively resolves the ambiguity of part granularity, allowing the model to produce reasonable part decompositions even without 2D mask guidance.

The second stage is **Diffusion-based Generation**. After obtaining the *blueprint*, we meticulously “forge” the high-fidelity geometry alongside the precise kinematic parameters promised in the planning stage. We innovatively propose a KineVoxel Injection (KVI) mechanism. This method cleverly encodes precise articulation parameters (like origin, axis, and limit) into a special *kinematic voxel*, allowing it to be jointly generated with the geometry-representing voxels during the diffusion denoising process—thereby achieving a synergistic synthesis of geometry and kinematic parameters.

To train our model effectively, we construct and introduce PhysDB, a large-scale dataset containing 150k assets. We define a novel four-tier annotation system that captures physics hierarchically. The holistic tier defines global properties like real-world scale and usage scene (e.g., kitchen, bedroom). The static properties tier covers part-level attributes such as semantic labels, physical materials (e.g., “metal”, “wood”), and mass. The functional tier defines part-level attributes such as intrinsic function (e.g., “to contain”) and state machines (e.g., [open, closed]). Finally, the interactive tier specifies kinematic properties, including joint types (e.g., revolute, prismatic), and atomic affordances (e.g., pushable, graspable).

PhysForge ultimately achieves the generation of functionally complete, physically interactive 3D assets from a single view image. Extensive experiments and qualitative demonstrations in physics simulator and game virtual world validate the effectiveness of the method, providing unprecedented high-fidelity, interactive assets for downstream applications such as robotic manipulation and game development.

Our core contributions are summarized as follows:

- **Formulation and Framework:** We propose a novel formulation for physics-grounded 3D generation, and a decoupled *VLM-based Planning + Diffusion-based Generation* two-stage framework (PhysForge).
- **Large-scale Dataset:** We contribute a large-scale, part-aware dataset with fine-grained, physical annotations (PhysDB), filling a critical data gap in the field.
- **Extensive Validation and Application:** We provide extensive experiments validating our framework’s SOTA performance on both planning and generation, and demonstrate the direct applicability of our assets in robotic simulators and interactive virtual worlds.

2. Related Work

2.1. 3D Content Generation

The field of 3D content generation has rapidly expanded, largely following two distinct philosophies: leveraging powerful 2D priors or training directly on 3D data. A foundational strategy, Score Distillation Sampling (SDS) pioneered by DreamFusion (Poole et al., 2023), enables text-to-3D synthesis without 3D supervision by optimizing a 3D representation using gradients from a 2D model. Significant progress has been made on 3D-native generation, with models such as 3DShape2VecSet (Zhang et al., 2023) introducing an encoding scheme that uses cross-attention for set-structured 3D data, CLAY (Zhang et al., 2024) scaling 3D diffusion to massive datasets, and TRELLIS (Xiang et al., 2024) introducing structured latents for a high-quality, coarse-to-fine generation process. Despite this rapid evolution in synthesizing high-fidelity geometry and textures, a common limitation unites all these approaches: the resulting assets are holistic and non-interactive.

2.2. Part-aware 3D Shape Generation

Recognizing the limitations of holistic generation, a recent line of work has begun to explore part-aware 3D generation. The central challenge in this sub-field is how to decompose a complex object into meaningful components while ensuring the final structure remains geometrically coherent. Early approaches have primarily adopted one of two strategies. The first is a “reconstruction-from-views” pipeline, which leverages 2D part masks to guide multi-view reconstruction (Liu et al., 2024a; Chen et al., 2024a). A significant advancement came from OmniPart (Yang et al., 2025), which introduced a two-stage framework built upon TRELLIS (Xiang et al., 2024) to achieve semantic decoupling and structural cohesion, enabling controllable part generation. Other approaches, like PartPacker (Tang et al., 2025), have focused on representation efficiency, compressing all parts into a compact dual volume representation for efficient generation from a single image. Critically, all these methods define *parts* based on purely geometric or visual boundaries. Their goal is to create assets that are visually decomposable. This leaves a crucial gap: the function and physics of a part are never considered.

2.3. Physics Grounded 3D Shape Generation

Recently, a few pioneering works have begun to bridge the gap between static geometry and interactive physics. PhysX-3D (Cao et al., 2025) makes a significant contribution by introducing PhysXNet, a dataset annotating physical properties on top of PartNet (Mo et al., 2019), and a generation model based on TRELLIS (Xiang et al., 2024) using a Physical VAE. Separate from holistic physics, another body of

research has focused specifically on articulation, a key component of interaction. This research has diverged into two main directions. One specialized direction has concentrated on the reconstruction of articulated objects, often termed “Digital Twins” (Liu et al., 2023; 2025b; Weng et al., 2024; Wu et al., 2025; Song et al., 2024; Tu et al., 2025). A second direction attempts procedural generation of articulated assets (Chen et al., 2024b; Gao et al., 2025; Le et al., 2024; Liu et al., 2024c;b; Mandi et al., 2024; Qiu et al., 2025). These approaches often rely on external, predefined content, such as part repositories, code templates, or VLM-predicted connectivity graphs, which constrains their ability to generalize to novel object categories and often leads to suboptimal accuracy.

3. Physics-Grounded, Part-Aware 3D Assets Generation

Our goal is to generate physics-grounded 3D assets that can serve a wide range of domains, from embodied AI simulation environments to interactive video games. To achieve this, our approach is built upon two pillars: (1) a comprehensive and diverse training dataset, and (2) a powerful and robust generation pipeline. We first introduce PhysDB, a novel large-scale dataset, in Section 3.1. It provides rich, fine-grained physical annotations necessary for this task. Following this, we introduce an innovative two-stage generation framework PhysForge, as shown in Figure 2. Stage 1 (Section 3.2) is a “VLM Planner” that generates a hierarchical physical blueprint. Stage 2 (Section 3.3) is a “Diffusion Realization” stage, which uses a novel KineVoxel Injection mechanism to synthesize high-fidelity geometry, texture and precise articulation parameters.

3.1. PhysDB: A Physics-Grounded Dataset

We propose a system of annotation that defines **holistic**, **static**, **functional**, and **interactive properties** to define the physical nature of each asset. At the object level, we define the asset’s real-world scale, its object category, and its intended usage scene (e.g., kitchen, bedroom). Descending to the part level, we first define static and semantic properties, such as the part’s semantic label, its physical material, and its mass. Next, we define functional properties inspired by OAKINK2 (Zhan et al., 2024), which include the part’s intrinsic function (e.g., “to contain”, “to control”) and its potential state machine (e.g., Button: [pressed, released]). Finally, our interactive tier specifies how an agent can interact with the object, detailing an atomic affordance library (e.g., pushable, rotatable) and, for movable parts, their complete kinematic definition: a parent part, a joint type (revolute, continuous, prismatic, or fixed), and the precise joint parameters (axis origin, direction, and limits).

We introduce PhysDB, a new dataset of 150k 3D objects

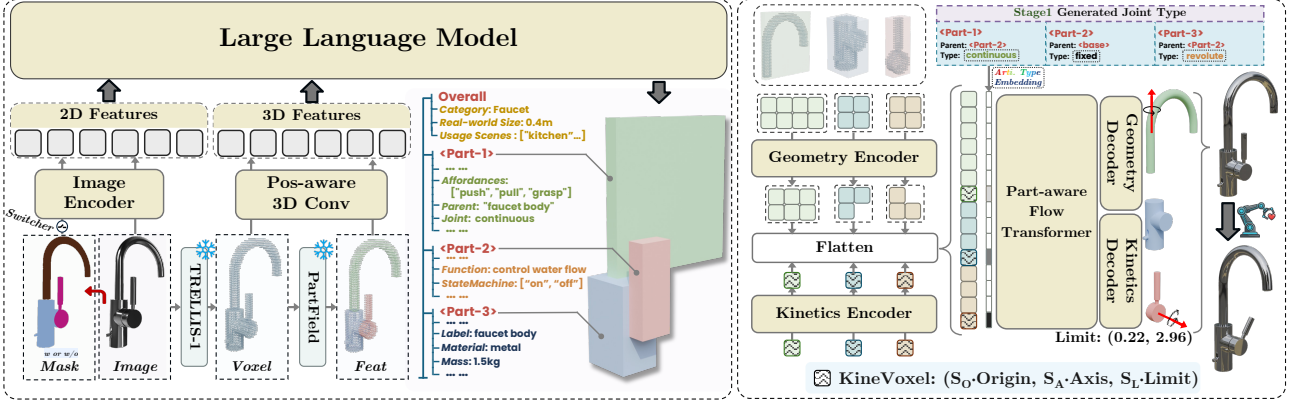


Figure 2. **Method overview.** PhysForge consists of two stages: (Left) Stage 1: VLM-based Planning, where the VLM planner generates a “Hierarchical Physical Blueprint” defining part structure and physical properties. (Right) Stage 2: Diffusion-based Generation, where a diffusion model, guided by the blueprint, uses the KineVoxel Injection (KVI) mechanism to synergistically generate the final geometry, texture, and precise kinematic parameters.

sourced from Objaverse (Deitke et al., 2023), covering seven major categories: household, industrial, weapons, personal, vehicles, tech & electronics, and cultural items. We select objects that are amenable to our physics annotation pipeline and already possess a meaningful part structure. Our annotation pipeline involves a human-in-the-loop process. We first render the whole objects and per-part images, which are fed to a multi modal LLM to generate initial annotations. This is followed by manual screening and correction to ensure the accuracy and consistency of the final PhysDB dataset. Scaling precise 3D articulation annotation to 150k objects is extremely challenging. Due to the wide variety of object categories, PhysDB focuses on providing rich physical properties and identifying joint types, rather than attempting to annotate precise numerical axes which are often inaccurate at this scale. To bridge this kinematic gap, we supplement our training process with PartNet-Mobility (Xiang et al., 2020) and Infinite-Mobility (Lian et al., 2025), which provide the ground-truth articulation parameters necessary to train our model in the diffusion stage.

3.2. VLM as a Physical Blueprint Planner

The VLM’s rich world knowledge provides a strong prior for object-part relationships, making it an ideal *planner* for our first stage. While VLMs lack explicit 3D understanding, we finetune them to evoke this capability. We select Qwen2.5-VL (Bai et al., 2025) as our base model due to its powerful knowledge base and vision capabilities. To integrate 3D information, the model accepts a single image I , its corresponding 3D voxel representation V (obtained from TRELIS (Xiang et al., 2024) first stage), and an optional 2D part mask M for granularity control. The input image I and the 2D mask M (which is converted to a color map) are processed directly by Qwen’s powerful image encoder. For the 3D voxel input V , we diverge from the common

3DShape2VecSet (Zhang et al., 2023) encoder. To better capture part-aware and local information, we first use a Part-Field encoder (Liu et al., 2025a) to extract features for each voxel, then apply a position-aware 3D convolutional network to downsample these features into a 512-dimensional voxel embedding.

With these encoded inputs, We finetune the VLM to autoregressively generate the complete part structure and physical properties. We introduce 66 new special tokens to the VLM’s codebook: `<boxs>` and `<boxe>` to delimit a bounding box, and 64 discrete tokens (`<box0>`, ..., `<box63>`) for the quantized coordinates. Each 3D axis-aligned bounding box is thus represented by only 6 tokens, enabling highly efficient structural planning. The model then outputs the hierarchical physical blueprint for each planned part. A key discovery is that physics-guided planning resolves part ambiguity. Training the model to co-predict physical properties (like material and function) alongside bounding boxes provides stronger semantic constraints. This synergy significantly improves the model’s understanding of part decomposition. As a result, even when no 2D mask is provided, the VLM can produce semantically coherent and reasonable bounding box plans.

3.3. Diffusion-based Generation with KineVoxel Injection

The VLM planner outputs a hierarchical structure, including per-part bounding boxes, parent-child relationships, and semantic joint types (e.g., fixed, revolute). While the VLM excels at this high-level structural and semantic planning, it is ill-suited for predicting the precise, continuous 3D values required for kinematics, such as an exact origin coordinate or axis vector. We therefore delegate this task to the diffusion head. This presents a challenge: how to synergistically generate these continuous parameters within a diffusion

Table 1. Quantitative comparison of Physics Property generation on the PhysXNet. Our method outperforms the baseline in both geometric generation quality and the accuracy of predicted physical properties.

Method	CD ↓	F1-0.1 ↑	F1-0.05 ↑	Absolute scale (cm) ↓	Material ↓	Affordance ↓	Description ↑
TRELLIS	10.10	86.53	72.47	-	-	-	-
PhysXGen	9.81	87.91	73.60	25.83	1.59	3.69	0.38
PhysForge (Ours)	9.21	89.24	75.43	11.04	0.81	1.22	0.87

Table 2. Quantitative comparison of Physics Property generation on the PhysDB. On our more diverse PhysDB dataset, our model demonstrates a more significant advantage over the baseline methods.

Method	CD ↓	F1-0.1 ↑	F1-0.05 ↑	Absolute scale (m) ↓	Material ↓	Function ↑	Interaction ↑
TRELLIS	24.32	68.19	53.28	-	-	-	-
PhysXGen	25.30	65.79	50.57	1.08	1.44	0.36	0.34
PhysForge (Ours)	22.89	70.51	55.38	0.37	0.43	0.83	0.96

Table 3. Quantitative results for bounding box generation (%) on PartObjaverse-Tiny. Results show that physics-guided planning significantly improves part planning accuracy and enables semantically reasonable results even without a 2D mask input.

Method	Voxel recall ↑	Voxel IoU ↑	Bbox IoU ↑
PartField	69.65	46.04	37.33
OmniPart (SAM mask)	68.33	43.34	34.33
PhysForge-bbox (w/o mask)	67.89	35.53	32.30
PhysForge (w/o mask)	73.63	47.66	36.32
OmniPart	73.79	52.92	41.66
PhysForge (Ours)	77.16	53.74	42.95

pipeline designed for geometry?

We solve this by extending the OmniPart (Yang et al., 2025) second stage framework with our novel KineVoxel Injection mechanism. Our approach begins by representing the articulation parameters for a single part i as an 8-dimensional vector $P_i = (O_i, A_i, L_i)$, where $O_i \in \mathbb{R}^3$ is the joint origin, $A_i \in \mathbb{R}^3$ is the joint axis, and $L_i \in \mathbb{R}^2$ is the motion limits. We represent P_i as a “KineVoxel”, a special representation that can be processed alongside the standard geometric latents Z_g in a unified denoising framework. Our approach maps data from different modalities (geometry and kinematics) into a unified latent space for joint diffusion. We utilize independent Kinematic Encoders (E_{kine}) and Decoders (D_{kine}) to process the KineVoxel, allowing it to share a latent space with the geometry latents within the middle transformer:

$$z_{k,i} = E_{kine}(\text{concat}(S_O \cdot O_i, S_A \cdot A_i, S_L \cdot L_i)),$$

where S_O, S_A, S_L are scaling factors. Both E_{kine} and D_{kine} are implemented as lightweight 2-layer MLPs. The diffusion network contains down-sample blocks, a middle transformer, and up-sample blocks. We inject our KineVoxel $z_{k,i}$ after downsampling, concatenating it with the sequence of geometry voxel latents $Z_g = \{z_{g,i}\}$ before they are fed into the main denoising transformer. To allow the transformer to distinguish between the two latent types, we add a joint type embedding E_{type} to the KineVoxel. This embedding E_{type} is derived from the VLM’s planned joint type (e.g., “revolute”) and is added to $z_{k,i}$. The transformer can thus learn the complex correlations between part geometry

and its corresponding joint parameters.

The entire model is trained by minimizing the Conditional Flow Matching (CFM) objective (Lipman et al., 2024). We define a composite loss that separates the contribution of geometry and kinematic voxels:

$$\mathcal{L} = \mathbb{E}_{t, Z_0, c} [\mathcal{L}_{geo} + \lambda_{kine} \cdot \mathcal{L}_{kine}]$$

where c is the condition from the VLM blueprint. The loss terms $\mathcal{L}_{geo}$ and $\mathcal{L}_{kine}$ are the standard L_2 losses between the predicted and target velocities for the geometry latents Z_g and kinematic latents Z_k , respectively:

$$\mathcal{L}_{geo} = \|v_{g,t} - \hat{v}_{g,t}\|^2; \mathcal{L}_{kine} = \|v_{k,t} - \hat{v}_{k,t}\|^2.$$

We set the weighting factor $\lambda_{kine} = 10$ throughout our training, placing a higher importance on accurately predicting the precise articulation parameters.

4. Experiments

Evaluation Protocol. To evaluate our model, we utilize the commonly used part-level dataset PartObjaverse-Tiny (Yang et al., 2024), which contains 200 diverse objects, and the test set (1000 objects) from PhysXNet (Cao et al., 2025). We also establish two new test sets: (1) a set of 1,000 cases sampled uniformly by category from our proposed PhysDB, and (2) a set of 340 articulated objects sampled from PartNet-Mobility and Infinite-Mobility. We first evaluate our model’s capability in the “Part Structure Planning via VLM” stage on the PartObjaverse-Tiny dataset, with results presented in Section 4.1. Following this, in Section 4.2, we evaluate the model’s performance on generating accurate physical properties and kinematic parameters. Finally, We demonstrate the broad applications of our model in Section 4.2.

4.1. Part Structure Planning

Baselines and Metrics. We first evaluate and analyze our model’s capability on the Part Structure Planning task. We select the first stage of OmniPart (Yang et al., 2025)

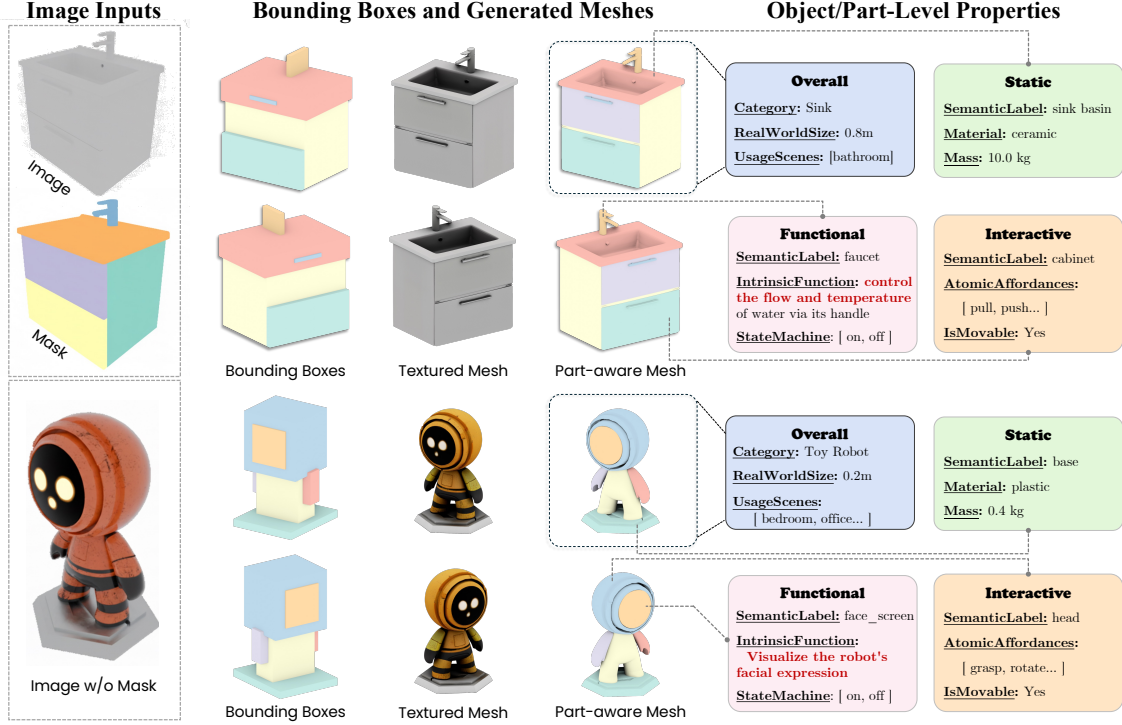


Figure 3. Qualitative results of PhysForge. Given a single image and an optional 2D mask for control, our model generates high-quality, physics-grounded, and part-aware 3D assets.

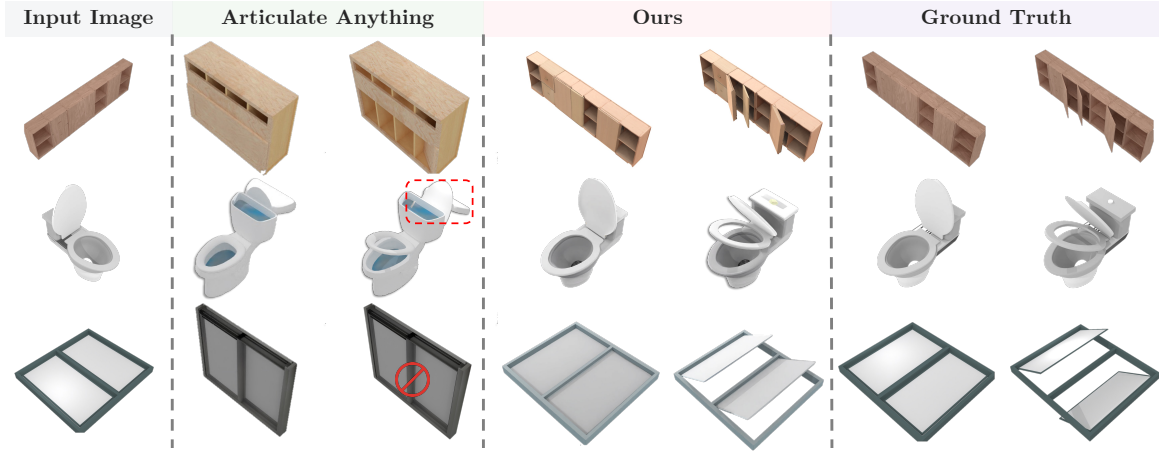


Figure 4. Qualitative results of articulated object generation from a single image.

and PartField (Liu et al., 2025a) as our primary baselines. The first stage of OmniPart stage trains an auto-regressive transformer on part-level data for bounding box generation, which, by default, requires a 2D mask input to control the granularity of the generated parts. PartField is a point cloud segmentation method that can also take voxels as input to produce voxel-level segmentation results and corresponding bounding boxes. As PartField requires the number of parts to define the segmentation scale, we provide the ground-truth number of parts as input. Following OmniPart, we use BBox IoU, Voxel Recall, and Voxel IoU as our evaluation

metrics, assessing both bounding box-level accuracy and voxel-level planning precision.

Results and Ablation Analysis. In Table 3, we show the comparison of all methods on the part structure planning task. To analyze our model’s planning ability with and without a 2D mask input, we introduce two additional experimental settings. The second row, “OmniPart (SAM mask)”, replaces OmniPart’s ground-truth 2D mask with a 2D mask obtained from SAM (Kirillov et al., 2023), filtering out small masks with an area ratio less than $1600/1024^2$. The third

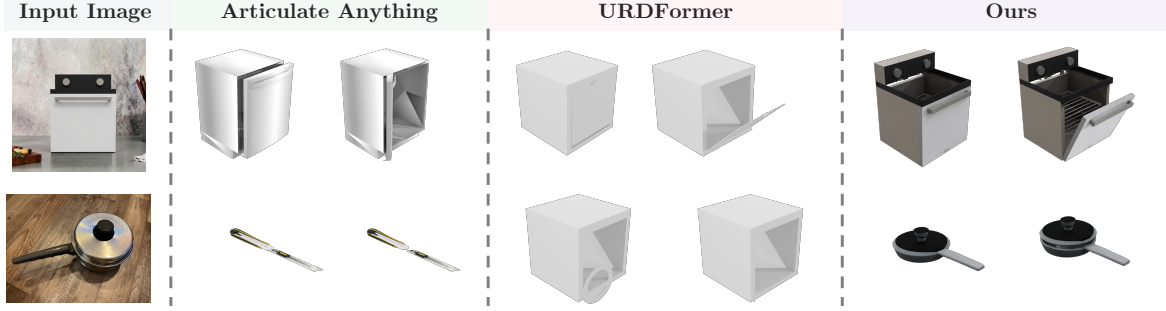


Figure 5. Qualitative results of articulated object generation from an in-the-wild image.

row, “PhysForge-bbox”, represents our model architecture trained only on the 500k part-level bounding box dataset (without physics). An entry marked “w/o mask” indicates that no mask was provided to the model input.

Comparing the overall results, our full model achieves state-of-the-art results, demonstrating the strongest part structure planning capability. The results of “PhysForge w/o mask” (row 4) are significantly better than the “PhysForge-bbox” model (row 3), which demonstrates that the introduction of physical properties significantly enhances our model’s semantic understanding and planning capabilities for part structures. Even without a mask input, it can still produce semantically reasonable results. Furthermore, our model operating without a mask still outperforms the OmniPart’s first stage that uses SAM-generated masks, highlighting the robustness of our physics-guided planning.

4.2. Physics-Grounded Generation

Baselines and Metrics. We evaluate our model’s ability to generate physics properties by selecting PhysXGen and TRELIS as our primary baseline. Specifically, we normalize the ground-truth and predicted shapes into a canonical space of $[-0.5, 0.5]$, then compute the Chamfer Distance (CD) and F1-Score. The F1-Score is assessed at two distance thresholds, $CD < 0.1$ and $CD < 0.05$. To evaluate the accuracy of physics properties at the part level, we compare the MAE of Absolute Scale, Material, Affordance, and the CLIP-Similarity of text-based Function and Interaction.

To evaluate our model’s performance in generating Kinematic Parameters, we select Articulate Anything (Le et al., 2024), Singapo and URDFormer as baselines, as they support articulated object generation from a single image. For this task, we use CD (%) and F-Score (%) to measure mesh generation quality, and CLIP-Similarity to assess the match with the input image. Following Articulate Anything, we utilize Joint Axis Error and Joint Pivot Error to measure the accuracy of the generated kinematic parameters. Specifically, we report Joint-Axis-Err-5 and Joint-Pivot-Err-5 on the subset of categories supported by all methods, and additionally report Joint-Axis-Err-all and Joint-Pivot-Err-all for

methods that generalize to all categories.

Physics Properties. In Table 1, we present a comparison of our method against the baselines PhysXGen and TRELIS. Our method surpasses the other methods in terms of geometry generation quality. Unlike PhysXGen, which is trained on specific categories and is limited to outputting opaque CLIP features, our method benefits from the VLM’s powerful world-knowledge prior, enabling it to directly and accurately output corresponding physics properties as both text and numerical values. Therefore, in the realm of physics property generation, our method significantly outperforms the baseline. Furthermore, in Figure 3, we demonstrate our model’s effectiveness in generating physics-grounded assets. From a single image, along with optional 2D mask control, our pipeline can accurately plan all part bounding boxes and physical attributes, and subsequently utilize the diffusion model to generate part-aware geometry and textures.

Kinematic Parameters. In Table 4, we present the quantitative comparison between our method and the baseline models, along with qualitative results on the validation set (Figure 4) and on in-the-wild images (Figure 5). The articulated objects generated by our method are significantly superior to the baseline in terms of both consistency with the input image and the accuracy of the joint parameters.

Ablation Analysis. We report the results of two key ablation studies in Table 4, which analyze the impact of removing the joint type embedding and the dedicated kinematic sub-network. The joint type embedding serves as the critical interface between our two stages: while Stage 1 predicts the qualitative articulation type (e.g., revolute, prismatic), this embedding provides a strong functional prior that constrains and guides the precise parameter estimation in Stage 2. The results clearly demonstrate that without the guidance from Stage 1’s planning, Stage 2 struggles to resolve kinematic ambiguities, leading to a degradation in joint accuracy, confirming that the joint type embedding is indispensable for effectively transferring physical common sense to the generation stage. Furthermore, removing the independent kinematic encoder and decoder further compromises the model’s ability to synthesize precise mechanical constraints.

Table 4. Quantitative comparison of articulated objects generation. Our method achieves higher fidelity to the input image and more accurate joint axis and pivot prediction.

Method	CD ↓	Clip-Sim ↑	Joint-Axis-Err-5 ↓	Joint-Pivot-Err-5 ↓	Joint-Axis-Err-all ↓	Joint-Pivot-Err-all ↓
Articulate Anything	23.31	0.87	0.608	0.257	0.694	0.197
Singapo	21.10	0.85	0.241	0.153	-	-
URDFormer	25.42	0.84	0.781	0.652	-	-
PhysForge (w/o joint type emb)	10.73	0.90	0.157	0.132	0.292	0.141
PhysForge (w/o kinetics enc)	11.31	0.89	0.158	0.117	0.204	0.120
PhysForge (Ours)	10.21	0.93	0.101	0.071	0.164	0.096

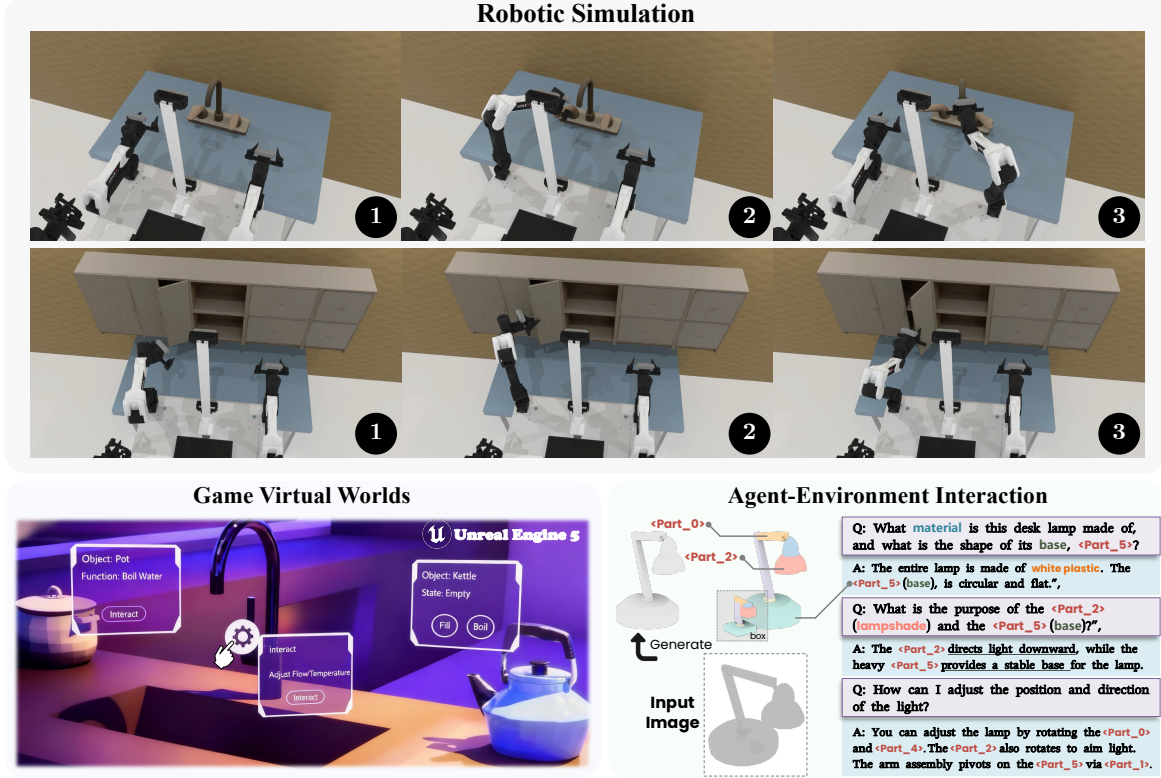


Figure 6. Downstream Applications of PhysForge. Our generated assets are simulation-ready: (a) A robotic arm manipulates an asset’s functional parts in a RoboTwin (Mu et al., 2025; Chen et al., 2025b) simulator. (b) The assets are imported into a virtual world (e.g., Unity/UE), enabling rich, physics-based interactions. (c) An agent interacts with our model via natural language, querying its physical blueprint to plan a task.

4.3. Application

To demonstrate the downstream utility of our generated assets, we showcase three primary applications in Figure 6: (a) Robotic Simulation: We demonstrate that our generated assets can be successfully imported into the RoboTwin (Mu et al., 2025; Chen et al., 2025b) simulation environment. The detailed part-level geometry and precise kinematic parameters allow robotic manipulators to realistically interact with the objects. (b) Virtual Worlds: In game engines and virtual worlds, our assets enable complex interactions. Because every part is endowed with physics-grounded attributes (materials, mass, articulation), developers can design sophisticated interaction logic without manual rigging. (c) Agent-Environment Interaction: Our VLM-based framework opens a new modality for interaction. An embodied agent (or VLA) can directly query our model in natural

language and receive a text-based physical blueprint with bounding boxes, providing an explicit plan for manipulation.

5. Conclusion

We introduce PhysForge, a novel framework that generates interactive and physics-grounded 3D assets. Our decoupled “VLM Planning + Diffusion Realization” architecture fine-tunes a VLM to generate “Hierarchical Physical Blueprints” that define an asset’s complete physical profile. To realize these blueprints, our KineVoxel Injection algorithm enables a diffusion model to synergistically generate geometry and precise kinematic parameters. This framework is supported by PhysDB, our large-scale, 150k-asset dataset with rich annotations. PhysForge provides a foundational data engine for embodied AI and interactive virtual worlds.

Impact Statements

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tang, J., et al. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- Cao, Z., Chen, Z., Pan, L., and Liu, Z. Physx-3d: Physical-grounded 3d asset generation. *arXiv preprint arXiv:2507.12465*, 2025.
- Chen, J., Xu, Z., Pan, X., Hu, Y., Qin, C., Goldstein, T., Huang, L., Zhou, T., Xie, S., Savarese, S., et al. Blip3-o: A family of fully open unified multimodal models-architecture, training and dataset. *arXiv preprint arXiv:2505.09568*, 2025a.
- Chen, M., Shapovalov, R., Laina, I., Monnier, T., Wang, J., Novotny, D., and Vedaldi, A. Partgen: Part-level 3d generation and reconstruction with multi-view diffusion models. *arXiv preprint arXiv:2412.18608*, 2024a.
- Chen, T., Chen, Z., Chen, B., Cai, Z., Liu, Y., Li, Z., Liang, Q., Lin, X., Ge, Y., Gu, Z., et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025b.
- Chen, Z., Walsman, A., Memmel, M., Mo, K., Fang, A., Vemuri, K., Wu, A., Fox, D., and Gupta, A. Urd-former: A pipeline for constructing articulated simulation environments from real-world images. *arXiv preprint arXiv:2405.11656*, 2024b.
- Deitke, M., Schwenk, D., Salvador, J., Weihs, L., Michel, O., VanderBilt, E., Schmidt, L., Ehsani, K., Kembhavi, A., and Farhadi, A. Objaverse: A universe of annotated 3d objects. In *CVPR*, 2023.
- Gao, D., Siddiqui, Y., Li, L., and Dai, A. Meshart: Generating articulated meshes with structure-guided transformers. In *CVPR*, 2025.
- Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W.-Y., et al. Segment anything. In *ICCV*, 2023.
- Le, L., Xie, J., Liang, W., Wang, H.-J., Yang, Y., Ma, Y. J., Vedder, K., Krishna, A., Jayaraman, D., and Eaton, E. Articulate-anything: Automatic modeling of articulated objects via a vision-language foundation model. *arXiv preprint arXiv:2410.13882*, 2024.
- Lian, X., Yu, Z., Liang, R., Wang, Y., Luo, L. R., Chen, K., Zhou, Y., Tang, Q., Xu, X., Lyu, Z., et al. Infinite mobility: Scalable high-fidelity synthesis of articulated objects via procedural generation. *arXiv preprint arXiv:2503.13424*, 2025.
- Lipman, Y., Chen, R. T., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. In *NeurIPS*, 2024.
- Liu, A., Lin, C., Liu, Y., Long, X., Dou, Z., Guo, H.-X., Luo, P., and Wang, W. Part123: part-aware 3d reconstruction from a single-view image. In *ACM SIGGRAPH*, 2024a.
- Liu, J., Mahdavi-Amiri, A., and Savva, M. Paris: Part-level reconstruction and motion analysis for articulated objects. In *ICCV*, 2023.
- Liu, J., Iliash, D., Chang, A. X., Savva, M., and Mahdavi-Amiri, A. Singapo: Single image controlled generation of articulated parts in objects. *arXiv preprint arXiv:2410.16499*, 2024b.
- Liu, J., Tam, H. I. I., Mahdavi-Amiri, A., and Savva, M. Cage: Controllable articulation generation. In *CVPR*, 2024c.
- Liu, M., Uy, M. A., Xiang, D., Su, H., Fidler, S., Sharp, N., and Gao, J. Partfield: Learning 3d feature fields for part segmentation and beyond. *arXiv preprint arXiv:2504.11451*, 2025a.
- Liu, Y., Jia, B., Lu, R., Ni, J., Zhu, S.-C., and Huang, S. Building interactable replicas of complex articulated objects via gaussian splatting. In *ICLR*, 2025b.
- Mandi, Z., Weng, Y., Bauer, D., and Song, S. Real2code: Reconstruct articulated objects via code generation. *arXiv preprint arXiv:2406.08474*, 2024.
- Mo, K., Zhu, S., Chang, A. X., Yi, L., Tripathi, S., Guibas, L. J., and Su, H. Partnet: A large-scale benchmark for fine-grained and hierarchical part-level 3d object understanding. In *CVPR*, 2019.
- Mu, Y., Chen, T., Chen, Z., Peng, S., Lan, Z., Gao, Z., Liang, Z., Yu, Q., Zou, Y., Xu, M., Lin, L., Xie, Z., Ding, M., and Luo, P. Robotwin: Dual-arm robot benchmark with generative digital twins. In *CVPR*, 2025.
- Poole, B., Jain, A., Barron, J. T., and Mildenhall, B. Dream-fusion: Text-to-3d using 2d diffusion. In *ICLR*, 2023.
- Qiu, X., Yang, J., Wang, Y., Chen, Z., Wang, Y., Wang, T.-H., Xian, Z., and Gan, C. Articulate anymesh: Open-vocabulary 3d articulated objects modeling. *arXiv preprint arXiv:2502.02590*, 2025.

- Song, C., Wei, J., Foo, C. S., Lin, G., and Liu, F. Reacto: Reconstructing articulated objects from a single video. In *CVPR*, 2024.
- Sun, Q., Cui, Y., Zhang, X., Zhang, F., Yu, Q., Wang, Y., Rao, Y., Liu, J., Huang, T., and Wang, X. Generative multimodal models are in-context learners. In *CVPR*, 2024.
- Tang, J., Lu, R., Li, Z., Hao, Z., Li, X., Wei, F., Song, S., Zeng, G., Liu, M.-Y., and Lin, T.-Y. Efficient part-level 3d object generation via dual volume packing. *arXiv preprint arXiv:2506.09980*, 2025.
- Tu, T., Li, M.-F., Lin, C. H., Cheng, Y.-C., Sun, M., and Yang, M.-H. Dreamo: Articulated 3d reconstruction from a single casual video. In *WACV*, 2025.
- Weng, Y., Wen, B., Tremblay, J., Blukis, V., Fox, D., Guibas, L., and Birchfield, S. Neural implicit representation for building digital twins of unknown articulated objects. In *CVPR*, 2024.
- Wu, D., Liu, L., Linli, Z., Huang, A., Song, L., Yu, Q., Wu, Q., and Lu, C. Reartgs: Reconstructing and generating articulated objects via 3d gaussian splatting with geometric and motion constraints. *arXiv preprint arXiv:2503.06677*, 2025.
- Xiang, F., Qin, Y., Mo, K., Xia, Y., Zhu, H., Liu, F., Liu, M., Jiang, H., Yuan, Y., Wang, H., et al. Sapien: A simulated part-based interactive environment. In *CVPR*, 2020.
- Xiang, J., Lv, Z., Xu, S., Deng, Y., Wang, R., Zhang, B., Chen, D., Tong, X., and Yang, J. Structured 3d latents for scalable and versatile 3d generation. *arXiv preprint arXiv:2412.01506*, 2024.
- Yang, Y., Huang, Y., Guo, Y.-C., Lu, L., Wu, X., Lam, E. Y., Cao, Y.-P., and Liu, X. Sampart3d: Segment any part in 3d objects. *arXiv preprint arXiv:2411.07184*, 2024.
- Yang, Y., Zhou, Y., Guo, Y.-C., Zou, Z.-X., Huang, Y., Liu, Y.-T., Xu, H., Liang, D., Cao, Y.-P., and Liu, X. Omnipart: Part-aware 3d generation with semantic decoupling and structural cohesion. *arXiv preprint arXiv:2507.06165*, 2025.
- Zhan, X., Yang, L., Zhao, Y., Mao, K., Xu, H., Lin, Z., Li, K., and Lu, C. Oakink2: A dataset of bimanual hands-object manipulation in complex task completion. In *CVPR*, 2024.
- Zhang, B., Tang, J., Niessner, M., and Wonka, P. 3dshape2vecset: A 3d shape representation for neural fields and generative diffusion models. *ACM Transactions On Graphics (TOG)*, 42(4):1–16, 2023.
- Zhang, L., Wang, Z., Zhang, Q., Qiu, Q., Pang, A., Jiang, H., Yang, W., Xu, L., and Yu, J. Clay: A controllable large-scale generative model for creating high-quality 3d assets. *ACM Transactions on Graphics (TOG)*, 43(4): 1–20, 2024.

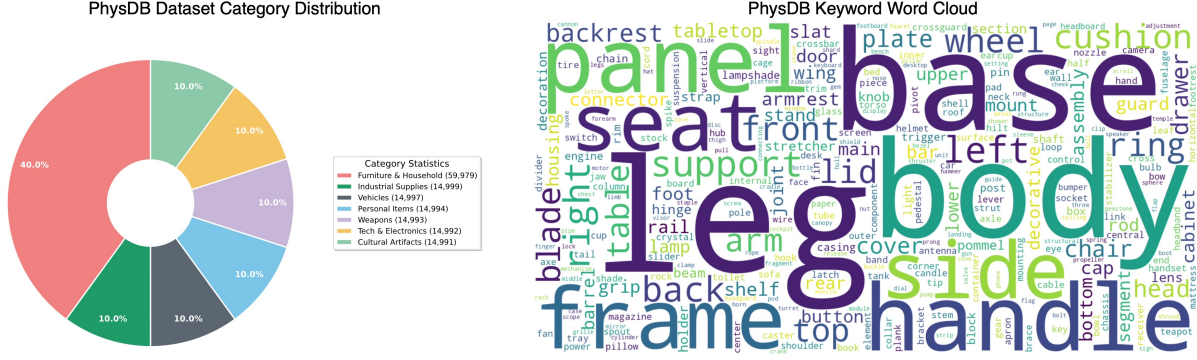


Figure 7. Analysis of major data categories and detailed label distribution.

A. Implementation Details

Training Setting. For the VLM-based Planning (Stage 1), we finetune our model based on Qwen2.5-VL (Bai et al., 2025) 3B. To ensure robust generalization and high-quality generation, we build PhysDB dataset, consisting of 150k objects with detailed, part-level physical annotations. Based on this, we construct 150k bounding box + physics property corpus entries and 150k physics-related Q&A entries. We also incorporate 500k additional objects with existing part segmentations to create a bounding box-only training set. Furthermore, we utilize the 36k objects from PhysXNet (Cao et al., 2025), generating an additional 36k training entries. Our final corpus totals 836k entries. During training, we apply balanced sampling based on the quantity of each data source. Simultaneously, we randomly drop the 2D mask input with a probability of 0.5 during training. For the first stage, we train our model for two days using 32 NVIDIA H20 GPUs.

For the Diffusion-based Generation (Stage 2), which trains the generation of geometry, texture, and kinematic parameters, we first include the widely-used PartNet-Mobility dataset (Xiang et al., 2020). To enhance the model’s generalization, we also utilize the Infinite-Mobility (Lian et al., 2025) method to construct a new set of 15k articulated objects across 20 categories. Furthermore, to ensure the generated geometry and texture maintain high visual fidelity, we incorporate an additional 50k part-aware objects that do not possess articulation information. This strategy helps sustain the model’s general part-aware generation capability. During training, these objects are assigned a unique, independent articulation embedding to effectively distinguish them from assets with specific articulation properties. We then finetune the second stage of the OmniPart (Yang et al., 2025) model using these combined data sources. For the second stage, we train our model for five days using 16 NVIDIA H20 GPUs.

Evaluation Setting. We evaluate physical property accuracy at the part level for consistent comparison. For the baseline PhysXGen (Cao et al., 2025), which treats each voxel as an independent element, we perform direct one-to-one matching between generated and ground-truth voxels. In contrast, since our method generates attributes based on functional components, we first match our predicted parts with GT parts to propagate part-level properties to the voxel grid.

On the PhysXNet dataset, we evaluate numerical attributes—including Absolute scale (cm), Material (represented by density), and Affordance—using Mean Absolute Error (MAE). For the “Description” attribute, as PhysXGen only yields CLIP features, we convert our generated text descriptions into CLIP embeddings to calculate CLIP similarity against the GT features.

On our proposed PhysDB dataset, which covers a wider array of categories and a vast range of Absolute scale (m), we employ similar metrics. For “Material”, we utilize the material-to-density correspondence defined in Table 5 to calculate numerical results for both the baseline and our method, reporting the MAE. Absolute scale is also evaluated via MAE. For the complex textual descriptions of “Function” and “Interaction”, we employ CLIP similarity to assess semantic alignment with the ground truth.

B. More Details of PhysDB

Analysis of PhysDB. We construct PhysDB by leveraging existing 3D assets equipped with part-level annotations. To ensure the dataset is suitable for the task of physical property estimation (e.g., material, friction, and articulation), we carefully filter the source data. We exclude categories where physical attributes are ambiguous or ill-defined (e.g., abstract

shapes or purely decorative plants) and prioritize categories with distinct, functional parts. To maintain a balanced yet representative distribution of real-world objects, we define specific ratios for different super-categories. Specifically, the dataset is composed of Furniture & Household (40%), followed by Industrial Supplies, Vehicles, Personal Items, Weapons, Tech & Electronics, and Cultural Artifacts, which each account for 10% of the total distribution. From the curated pool, we establish a standardized test set containing 1,000 objects. We employ a stratified sampling strategy based on the category ratios defined above to ensure the test set distribution mirrors the overall dataset. Furthermore, within each category stratum, we select instances based on a quality score—prioritizing meshes with higher geometric integrity and clearer part definitions—to ensure a reliable and rigorous evaluation benchmark. Figure 7 presents the distribution of major data categories as well as detailed statistics of individual labels in our dataset.

Data Filtering and Annotation Pipeline. The construction of PhysDB involves a rigorous multi-stage filtering process to ensure data quality. We first select object categories from Objaverse that are conducive to physical property annotation, subsequently filtering out low-quality or overly simplistic geometries. Based on initial component decomposition, we exclude objects with either a single part or more than 16 parts to maintain a reasonable complexity for physical reasoning. For the remaining candidates, we implement an automated part-merging strategy that combines adjacent or extremely small components based on the IoU values of their bounding boxes. This is followed by a VLM-based screening and manual spot-checks to eliminate unreasonable merges, ensuring only objects with semantically clear and physically meaningful part divisions are retained.

For the annotation stage, we generate high-quality renderings for each individual part alongside a holistic view of the entire object. The holistic viewpoints are strategically chosen to either maximize the visible area of the specific part or to provide a standard frontal perspective, ensuring sufficient contextual information for VLM. After the initial generation of all physical attributes, we perform a final round of human review to identify and filter out any problematic or inconsistent annotations, resulting in a reliable and simulation-ready dataset.

C. More Results

Physical Properties. We present additional results of mesh and physical property generation using our model. In Figure 8, we show how the input of a 2D mask enables corresponding bounding box planning, physical property estimation, and the generation of mesh geometry and texture. In Figure 9, we demonstrate results produced by our first stage planning approach without the use of a 2D mask.

Articulated Objects. In Figure 10, we present a wider variety of articulated objects generated by our model.

Interactive Virtual World and Robotic Simulation. We have included demo videos showcasing Interactive Virtual World and Robot Simulation in the supplementary materials.

D. Construction of Training Corpus

When constructing the training data for the first-stage VLM, we design various question patterns to enhance the model’s generalization ability. We use special tokens to represent bounding boxes, and format physical properties and object hierarchies as JSON outputs. This allows for direct parsing of VLM outputs and facilitates subsequent generation in the second stage. Examples of our constructed training corpus are shown in Table 7 and Table 8.

Table 5. Physical parameters for various material categories used in FEM simulations. The parameters include density (ρ) in kg/m^3 , Young’s modulus (E) in Pa, Poisson’s ratio (ν), and Rayleigh damping coefficients (α and β).

Material	ρ (kg/m^3)	E (Pa)	ν	α	β
Fabric	550	2.1×10^9	0.25	125	3.0×10^{-5}
Stone	2700	5.5×10^{10}	0.25	190	1.9×10^{-6}
Ceramic	3800	3.7×10^{11}	0.22	6	1.0×10^{-7}
Bone	900	1.75×10^{10}	0.325	1	2.0×10^{-6}
Shell	2800	6.0×10^{10}	0.3	6	1.0×10^{-7}
Metal	2700	6.9×10^{10}	0.33	5	1.0×10^{-7}
Wood	750	1.1×10^{10}	0.35	60	2.0×10^{-6}
Plastic	940	3.0×10^8	0.47	30	1.0×10^{-6}
Leather	1050	7.0×10^7	0.7	30	1.0×10^{-8}
Wax	900	4.5×10^8	0.4	125	1.0×10^{-5}
Rubber	935	1.0×10^8	0.49	550	1.0×10^{-4}
Vegetable	1000	5.0×10^4	0.45	30	1.0×10^{-7}
Glass	2500	7.0×10^{10}	0.22	1	1.0×10^{-7}
Paper	750	5.0×10^7	0.4	50	1.0×10^{-5}
Chocolate	1200	2.0×10^9	0.35	160	3.0×10^{-6}
Chalk	1400	3.0×10^9	0.25	540	9.0×10^{-6}
Mushroom	950	5.0×10^4	0.45	91.6	2.0×10^{-3}
Clay	2000	3.5×10^{10}	0.2	115	3.6×10^{-6}
Candy	1400	5.0×10^9	0.3	500	1.6×10^{-6}
Plaster	1200	4.0×10^9	0.25	520	6.6×10^{-6}
Bronze	8800	1.1×10^{11}	0.34	7	1.0×10^{-7}
Fossil	2700	5.5×10^{10}	0.25	190	1.9×10^{-6}
Skull	1600	1.5×10^{10}	0.3	330	3.3×10^{-6}
Rock	2500	5.0×10^{10}	0.25	100	5.0×10^{-6}
Iron	8000	2.1×10^{11}	0.28	10	1.0×10^{-7}
Polycarbonate	1190	2.2×10^9	0.37	0.5	4.0×10^{-7}
Steel	7850	2.0×10^{11}	0.29	20	3.0×10^{-8}
Tin	7265	5.0×10^{10}	0.325	2	3.0×10^{-8}

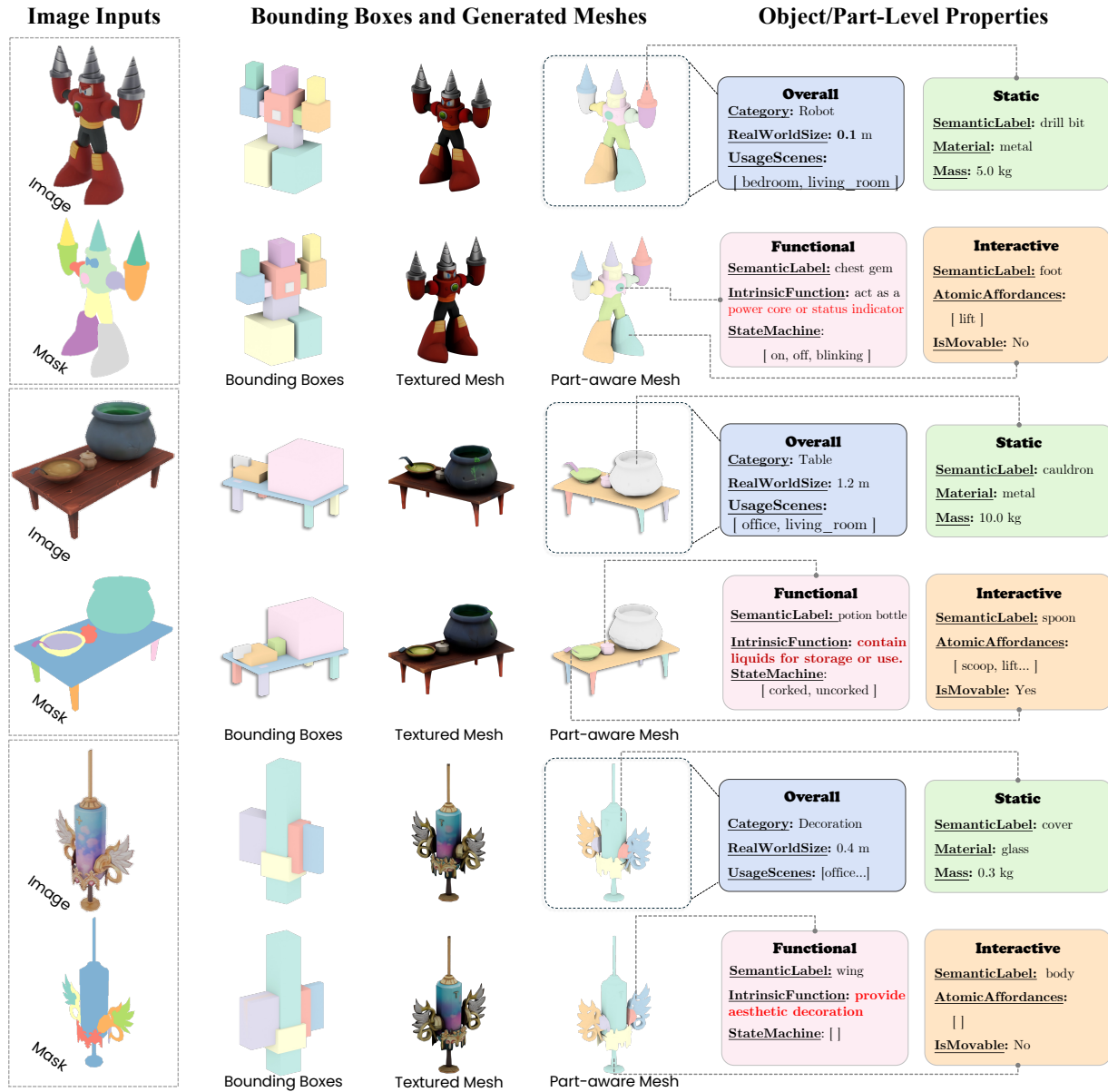


Figure 8. Qualitative results of mesh and physical properties generation from a single image, with a 2D mask used for control.

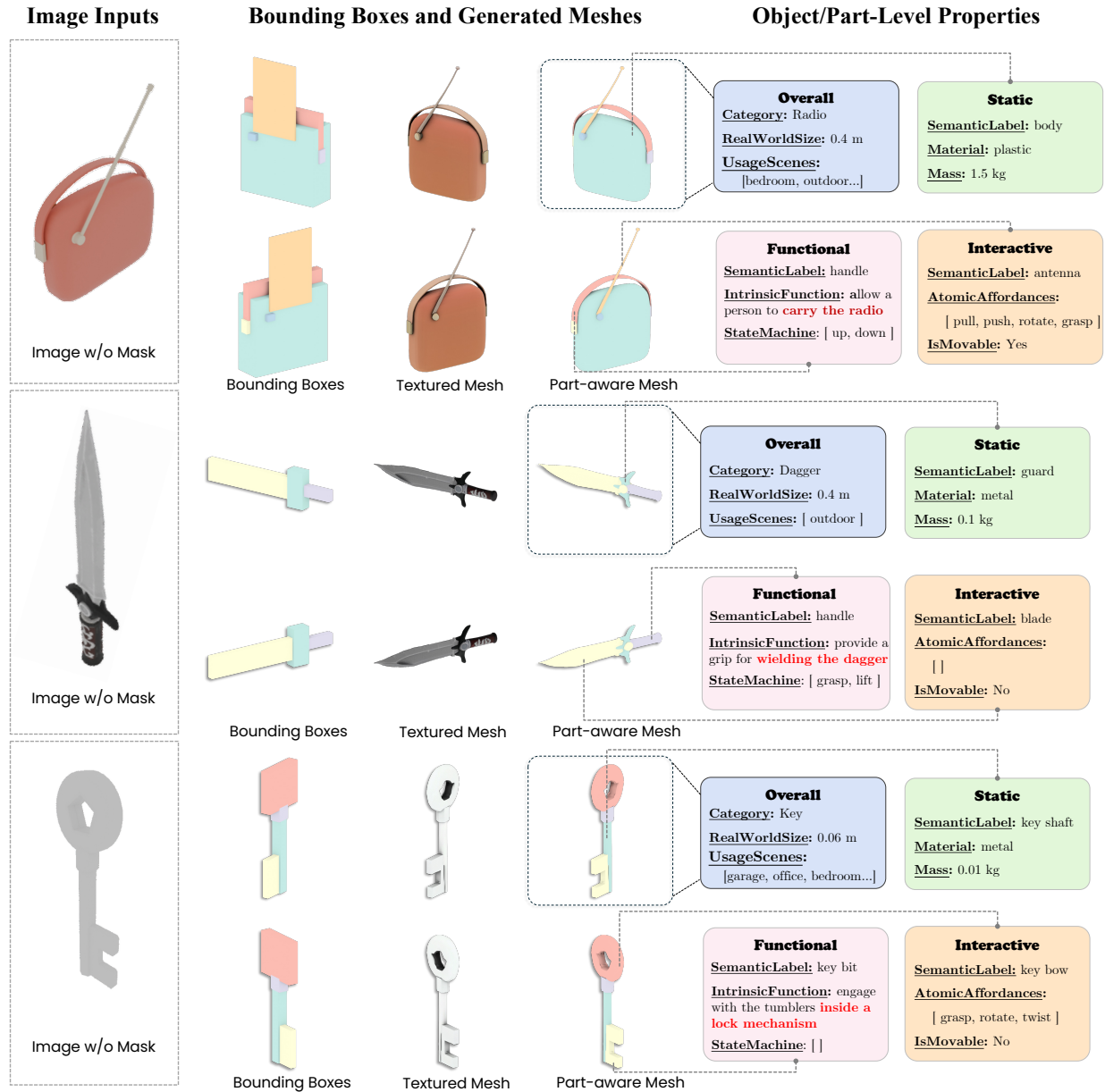


Figure 9. Qualitative results of mesh and physical properties generation from a single image.

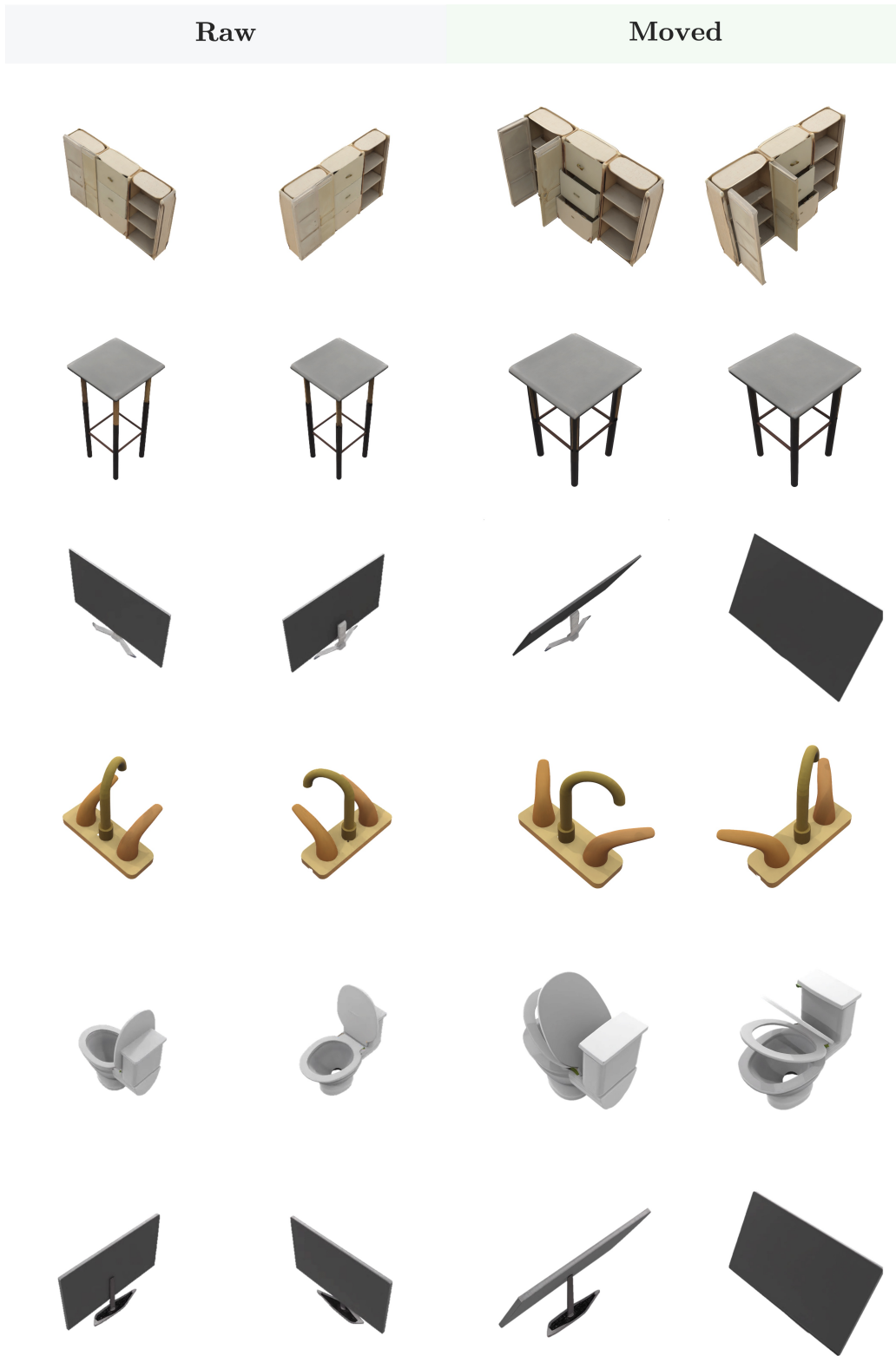


Figure 10. Qualitative results of articulated objects generation.

Table 6. **Training data example.** The input involves multimodal prompts including physics properties and bounding boxes requests. The output demonstrates a structured response containing token sequences and a detailed hierarchical JSON.



Input: Generate one JSON: bounding_boxes plus physics_properties (material, mass.kg, friction μ_s/μ_k , mobility, joint type+parent_id, affordances).

<voxel> This is a rendered image of this 3D object from one view: <image>.

Output:

```
'<boxs><box-0><box-24><box-0><box-63><box-49><box-54><boxe><boxs><box-4><box-14><box-39><
box-19><box-38><box-63><boxe><boxs><box-44><box-14><box-39><box-59><box-38><box-63><
boxe>'

{
  "ObjectLevel": {
    "OverallCaption": "A pair of simple black on-ear headphones for listening to audio.",
    "Category": "Headphones",
    "RealWorldSize": 0.2,
    "UsageScenes": [
      "office",
      "living_room",
      "bedroom",
      "outdoor"
    ]
  },
  "PartLevel": [
    {
      "PartIndex": 0,
      "SemanticLabel": "headband",
      "Q2": "A curved black plastic band designed to be worn over the head.",
      "Material": "plastic",
      "Mass": 0.1,
      "IntrinsicFunction": "Connect the two earcups and secure the headphones on a user's
      head.",
      "StateMachine": [],
      "SuitableForInteraction": "No",
      "AtomicAffordances": [],
      "IsMovable": "No"
    },
    {
      "PartIndex": 1,
      "SemanticLabel": "left_earcup",
      "Q2": "A circular black plastic earcup with a central depression for the ear.",
      "Material": "plastic",
      "Mass": 0.05,
      "IntrinsicFunction": "Cover the left ear and direct sound into it.",
      "StateMachine": [],
      "SuitableForInteraction": "No",
      "AtomicAffordances": [],
      "IsMovable": "Yes"
    },
    {
      "PartIndex": 2,
      "SemanticLabel": "right_earcup",
      "Q2": "A circular black plastic earcup with a central depression for the ear.",
      "Material": "plastic",
      "Mass": 0.05,
      "IntrinsicFunction": "Cover the right ear and direct sound into it.",
```

```
93547     "StateMachine": [],
93648     "SuitableForInteraction": "No",
93749     "AtomicAffordances": [],
93850     "IsMovable": "Yes"
93951   }
94052 ],
94153 "PartJointRelationships": [
94254   { "PartIndex": 0, "ParentPart": "world", "JointType": "fixed" },
94355   { "PartIndex": 1, "ParentPart": "<part-0>", "JointType": "revolute" },
94456   { "PartIndex": 2, "ParentPart": "<part-0>", "JointType": "revolute" }
94557 ]
94658 }
```

Table 7. **Training data example.** The input involves multimodal prompts including physics properties and bounding boxes requests. The output demonstrates a structured response containing token sequences and a detailed hierarchical JSON.



Input: Generate one JSON: bounding_boxes plus physics_properties (material, mass.kg, friction μ_s/μ_k , mobility, joint type+parent_id, affordances).

<voxel> This is a rendered image of this 3D object from one view: <image>.

Output:

```
'<boxs><box-12><box-21><box-0><box-53><box-42><box-63><boxe><boxs><box-35><box-22><box-0><
box-52><box-41><box-13><boxe><boxs><box-33><box-30><box-22><box-38><box-33><box-31><
boxe><boxs><box-10><box-20><box-37><box-21><box-43><box-63><boxe>'

{
  "ObjectLevel": {
    "OverallCaption": "A handheld, gun-shaped infrared thermometer with a digital display",
    "Category": "Infrared Thermometer",
    "RealWorldSize": 0.2,
    "UsageScenes": [
      "office",
      "home",
      "garage"
    ]
  },
  "PartLevel": [
    {
      "PartIndex": 0,
      "SemanticLabel": "sensor_housing",
      "Q2": "Dark grey plastic housing at the front, containing the sensor.",
      "Material": "plastic",
      "Mass": 0.02,
      "IntrinsicFunction": "house and protect the infrared sensor",
      "StateMachine": [],
      "SuitableForInteraction": "No",
      "AtomicAffordances": [],
      "IsMovable": "No"
    },
    {
      "PartIndex": 1,
      "SemanticLabel": "trigger",
      "Q2": "A grey, curved plastic trigger located on the handle.",
      "Material": "plastic",
      "Mass": 0.01,
      "IntrinsicFunction": "activate the temperature measurement",
      "StateMachine": [
        "pressed",
        "released"
      ],
      "SuitableForInteraction": "Yes",
      "AtomicAffordances": [
        "press"
      ],
      "IsMovable": "Yes"
    },
    {
      "PartIndex": 2,
      "SemanticLabel": "main_body",
```

```

10447   "Q2": "Light grey, gun-shaped plastic body with a handle and display area.",
10448   "Material": "plastic",
10449   "Mass": 0.15,
10450   "IntrinsicFunction": "house internal electronics and provide a grip",
10451   "StateMachine": [],
10452   "SuitableForInteraction": "Yes",
10453   "AtomicAffordances": [
10454     "grasp"
10455   ],
10456   "IsMovable": "No"
10457 },
10458 {
10459   "PartIndex": 3,
10460   "SemanticLabel": "battery_cover",
10461   "Q2": "Dark grey plastic cover at the bottom of the handle.",
10462   "Material": "plastic",
10463   "Mass": 0.02,
10464   "IntrinsicFunction": "cover and secure the battery compartment",
10465   "StateMachine": [
10466     "open",
10467     "closed"
10468   ],
10469   "SuitableForInteraction": "Yes",
10470   "AtomicAffordances": [
10471     "slide"
10472   ],
10473   "IsMovable": "Yes"
10474 }
10475 ],
10476 "PartJointRelationships": [
10477   { "PartIndex": 0, "ParentPart": "<part-2>", "JointType": "fixed" },
10478   { "PartIndex": 1, "ParentPart": "<part-2>", "JointType": "prismatic" },
10479   { "PartIndex": 2, "ParentPart": "world", "JointType": "fixed" },
10480   { "PartIndex": 3, "ParentPart": "<part-2>", "JointType": "prismatic" }
10481 ]
10482 }

```

Table 8. **Training data example.** The input involves multimodal prompts requesting bounding boxes and physics properties. The output contains token sequences and a structured JSON description of a toy rocket.



Input: Return one JSON: bounding_boxes and physics per object (material, mass.kg, friction {mu_s,mu_k}, mobility, joint {type,parent}, affordances).

<voxel> This is a rendered image of this 3D object from one view, <image> This is the mask of this rendered image, <image>

Output:

```
'<boxs><box-17><box-30><box-0><box-26><box-40><box-10><boxe><boxs><box-37><box-30><box-0><box-46><box-40><box-10><boxe><boxs><box-22><box-25><box-6><box-41><box-42><box-63><boxe><boxs><box-35><box-28><box-8><box-48><box-41><box-20><boxe><boxs><box-15><box-28><box-9><box-28><box-41><box-20><boxe><boxs><box-27><box-21><box-37><box-36><box-30><box-46><boxe><boxs><box-29><box-21><box-39><box-34><box-26><box-44><boxe><boxs><box-27><box-21><box-41><box-36><box-30><box-47><boxe>'

{
  "ObjectLevel": {
    "OverallCaption": "A simple toy rocket with a pointed nose and side boosters.",
    "Category": "Toy Rocket",
    "RealWorldSize": 0.3,
    "UsageScenes": ["bedroom"]
  },
  "PartLevel": [
    {
      "PartIndex": 0,
      "SemanticLabel": "booster_fin",
      "Material": "plastic",
      "Mass": 0.05,
      "IntrinsicFunction": "Provide aerodynamic stability and support the booster.",
      "StateMachine": [],
      "SuitableForInteraction": "No",
      "AtomicAffordances": [],
      "IsMovable": "No",
      "Caption": "A faceted, shield-shaped fin with a pointed bottom."
    },
    {
      "PartIndex": 1,
      "SemanticLabel": "booster_fin",
      "Material": "plastic",
      "Mass": 0.05,
      "IntrinsicFunction": "Provide aerodynamic stability and support the booster.",
      "StateMachine": [],
      "SuitableForInteraction": "No",
      "AtomicAffordances": [],
      "IsMovable": "No",
      "Caption": "A faceted, shield-shaped fin with a pointed bottom."
    },
    {
      "PartIndex": 2,
      "SemanticLabel": "rocket_body",
      "Material": "plastic",
      "Mass": 0.5,
      "IntrinsicFunction": "Form the main structural body and nose of the rocket.",
      "StateMachine": [],
      "SuitableForInteraction": "Yes",
      "AtomicAffordances": ["grasp", "lift"],
```

```

1144     "IsMovable": "No",
1145     "Caption": "A long, teardrop-shaped fuselage with a pointed nose cone."
1146   },
1147   {
1148     "PartIndex": 3,
1149     "SemanticLabel": "booster_body",
1150     "Material": "plastic",
1151     "Mass": 0.1,
1152     "IntrinsicFunction": "Form the main structure of the side booster.",
1153     "StateMachine": [],
1154     "SuitableForInteraction": "No",
1155     "AtomicAffordances": [],
1156     "IsMovable": "No",
1157     "Caption": "A short, cylindrical body with a rounded top edge."
1158   },
1159   {
1160     "PartIndex": 4,
1161     "SemanticLabel": "booster_body",
1162     "Material": "plastic",
1163     "Mass": 0.1,
1164     "IntrinsicFunction": "Form the main structure of the side booster.",
1165     "StateMachine": [],
1166     "SuitableForInteraction": "No",
1167     "AtomicAffordances": [],
1168     "IsMovable": "No",
1169     "Caption": "A short, cylindrical body with a rounded top edge."
1170   },
1171   {
1172     "PartIndex": 5,
1173     "SemanticLabel": "booster_nozzle",
1174     "Material": "plastic",
1175     "Mass": 0.02,
1176     "IntrinsicFunction": "Represent the engine exhaust nozzle of the booster.",
1177     "SuitableForInteraction": "No",
1178     "Caption": "A flat, circular disc representing the engine nozzle."
1179   },
1180   {
1181     "PartIndex": 6,
1182     "SemanticLabel": "booster_nozzle",
1183     "Material": "plastic",
1184     "Mass": 0.02,
1185     "IntrinsicFunction": "Represent the engine exhaust nozzle of the booster.",
1186     "SuitableForInteraction": "No",
1187     "Caption": "A flat, circular disc representing the engine nozzle."
1188   },
1189   {
1190     "PartIndex": 7,
1191     "SemanticLabel": "window",
1192     "Material": "plastic",
1193     "Mass": 0.03,
1194     "IntrinsicFunction": "Represent a viewing window on the rocket body.",
1195     "SuitableForInteraction": "No",
1196     "Caption": "A smooth, hemispherical dome representing a window."
1197   }
1198 ],
1199 "PartJointRelationships": [
1200   {"PartIndex": 0, "ParentPart": "<part-3>", "JointType": "fixed"},
1201   {"PartIndex": 1, "ParentPart": "<part-4>", "JointType": "fixed"},
1202   {"PartIndex": 2, "ParentPart": "world", "JointType": "fixed"},
1203   {"PartIndex": 3, "ParentPart": "<part-2>", "JointType": "fixed"},
1204   {"PartIndex": 4, "ParentPart": "<part-2>", "JointType": "fixed"},
1205   {"PartIndex": 5, "ParentPart": "<part-4>", "JointType": "fixed"},
1206   {"PartIndex": 6, "ParentPart": "<part-3>", "JointType": "fixed"},
1207   {"PartIndex": 7, "ParentPart": "<part-2>", "JointType": "fixed"}
1208 ]
1209

```

12109 }

1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264